

07 — Frontend Test Results and AI Report UI

Product: Endpoint IQ **Document:** Test Results and AI Report UI

This is the highest-value part of the frontend: it's where Endpoint IQ's core promise — turning API test failures into developer-actionable information — is delivered. Every element here should help a developer answer, in order: **What failed? Why? What's probably causing it? How do I fix it? What should I check in my code?**

1. Flow Overview

```
flowchart TD
  A[Test Run] --> B[Progress]
  B --> C[Results Overview]
  C --> D[Failed Endpoints List]
  D --> E[Failed API Detail]
  E --> F[AI Analysis]
  F --> G[Root Cause]
  G --> H[Recommended Fix]
  H --> I[Developer Fix Prompt]
  C --> J[Final Report]
  I --> J
```

2. Test Run Dashboard

Lives at `/projects/:projectId/test-runs/:runId` (see 04-...). Shows: - Run metadata: start time, duration, status (StatusPill). - Live progress bar while status === "running" (jobs completed / total, current endpoint being tested — sourced from the WS channel, §16 of 05-Frontend-API-Integration.md). - Once complete, a CTA to view full Results.

3. Overall Statistics

A stats strip/card row at the top of the Results page:

Stat	Source
Total endpoints tested	<code>test_results.count</code> for the run
Passed	<code>count where outcome === "passed"</code>
Failed	<code>count where outcome === "failed"</code>
Warnings	<code>count where outcome === "warning"</code>
Avg. response time	<code>avg of responseTimeMs</code>
Roles tested	<code>distinct roleId in results</code>

Rendered as compact stat cards, not a chart-heavy dashboard (per the “no excessive animation, DevOps-tool feel” design principle in 03-UI-UX-Specification.md).

4. Passed / Failed Endpoints

A toggle or filter at the top of the results table: **All / Passed / Failed / Warnings**, defaulting to a view that surfaces failures first (e.g. failures sorted to top, or the “Failed” tab pre-selected when failures exist).

5. Endpoint Result Table

Primary component: `EndpointResultTable`.

Column	Notes
Method	<code>MethodBadge</code>
Path	<code>Monospace</code>
Role	Which role the test ran as
Status	<code>StatusPill</code> (Passed/Failed/Warning)
Status Code	Actual HTTP status returned
Response Time	ms
Severity	Only populated for failures (<code>StatusPill</code> critical variant if security-related)

Row click → opens **Failed API Details** (for failures) or a lighter read-only detail (for passes).

6. Failed API Details

Component: `FailedEndpointDetail` — typically a slide-over panel or dedicated nested route (`.../results/:resultId`) so the table context is preserved. Composed of tabs:

flowchart LR

T1[Overview] --> T2[Request] --> T3[Response] --> T4[Diff] --> T5[AI Analysis]

- **Overview tab:** endpoint, role, status code (expected vs. actual), response time, severity, one-line error summary.
- **Request tab:** method, path, headers, body — via `RequestResponseViewer` (built on `JsonViewer/CodeBlock`).
- **Response tab:** status, headers, body — same viewer.
- **Diff tab:** expected vs. actual response, rendered as a structured diff (highlighting added/removed/changed fields), not a raw text diff, since responses are typically JSON.
- **AI Analysis tab:** see §15 below.

7. Request Viewer

Shows method, full path (with resolved path params), headers (collapsible), and body (pretty-printed JSON via `JsonViewer`, syntax highlighted, copyable).

8. Response Viewer

Same pattern as Request Viewer: status line, headers, body. Body rendering must handle non-JSON responses (plain text, HTML error pages) gracefully — fall back to a plain `CodeBlock`.

9. Headers Viewer

Shared sub-component of both Request and Response viewers: collapsible key-value list, monospace, with a “copy all” action.

10. Status Code

Rendered prominently with color coding (2xx green, 4xx amber, 5xx red) and, for failures, an explicit **Expected: 200 · Actual: 500** style comparison line.

11. Response Time

Displayed in ms, with a subtle visual flag (not alarming) if it's a significant outlier relative to the endpoint's other role results — this is a nice-to-have, not required for MVP.

12. Expected vs. Actual Result

The core of the Diff tab: side-by-side or unified structured diff of expected response schema/values (derived from the OpenAPI spec) vs. the actual response body received.

13. Error Details

If the target API returned an error body, its message/code is surfaced directly (not just the HTTP status). Network-level errors (timeout, connection refused) are shown distinctly from application-level errors (4xx/5xx with a body), consistent with §10 Risks in the architecture (“Target API Down... Workers time out... Retry with exponential backoff; mark job failed after N attempts”) — the UI should indicate if a result reflects a retried/exhausted-retries failure.

14. Response Differences

Structured JSON diff component, reused from the Diff tab (§12) — implemented once in components/ (or features/test-results/components/) since it's used both in the failed-detail view and potentially in report previews.

15. Severity

A severity badge (low/medium/high/critical) shown on the result row and in the failed detail overview. `critical` is reserved for AI-flagged security findings (see §19) and is visually distinct from a routine functional failure, per 03-UI-UX-Specification.md §21.

16. AI Analysis

Component: `AIAnalysisCard`, shown both inline (within Failed API Details, scoped to that one result) and at the run level (aggregated findings — missing coverage, security summary). Always carries the “AI-generated” tag and a muted disclaimer treatment, per the explicit product requirement that **AI recommendations are suggestions for review, not an unquestionable authority**.

17. Root Cause

A short, plain-language paragraph from `AIAnalysis.rootCause`. Rendered as the first block inside `AIAnalysisCard`, since it directly answers “Why did it fail?”

18. Recommended Solution

`AIAnalysis.recommendedFix`, rendered below Root Cause. Framed as a suggestion (“Consider checking...”, “This may be caused by...”) in copy tone, reinforcing it is not an automated fix.

19. Missing Test Coverage

Run-level (not per-result) — a list (`AIAnalysis.missingCoverage`) shown in a dedicated “AI Analysis” tab/section at the run level (e.g. `/projects/:projectId/results` → “AI Analysis” sub-tab), separate from individual failed-endpoint analysis.

20. Security Findings (if available)

Run-level list of `SecurityFinding` items, each with title, description, and severity. Rendered with the critical/high severity treatment prominently, consistent with the architecture’s explicit mention of AI-detected “security concerns” (§4.8) and the product instruction to surface security findings when available (not guaranteed to always be present — must have a graceful “no findings” state, not an empty/broken one).

21. Developer Fix Prompt

A distinct, copyable block (`AIAnalysis.developerPrompt`) — a ready-to-paste prompt/summary the developer can hand to their own coding tool. Rendered in a `CodeBlock`-style container with a prominent “Copy” button. This directly serves the product’s goal of turning failures into actionable next steps.

22. Report Download

Final section/page (`/projects/:projectId/reports/:reportId`, see 04-...): summary stats mirroring §3 above, AI recommendations highlighted at the top (per §4.9 of the architecture: “AI recommendations highlighted at the top”), and a prominent Download button. The report view is a condensed, read-only echo of the Results + AI Analysis UI — it should reuse the same `StatusPill`, `AIAnalysisCard`, and stats components rather than reimplementing them, for visual and code consistency.

23. Component Hierarchy

```
ResultsPage
├── ResultsSummaryStats
├── ResultsFilterBar (status/method/role filters)
├── EndpointResultTable
│   └── EndpointResultRow (→ opens FailedEndpointDetail)
├── FailedEndpointDetail (panel/route)
│   └── Tabs
│       ├── OverviewTab
│       ├── RequestTab → RequestResponseViewer
│       ├── ResponseTab → RequestResponseViewer
│       ├── DiffTab → ResponseDiffViewer
│       └── AIAnalysisTab → AIAnalysisCard
│           ├── RootCauseBlock
│           ├── RecommendedFixBlock
│           └── DeveloperPromptBlock (copyable)
├── RunLevelAIAnalysisSection
│   ├── MissingCoverageList
│   ├── SecurityFindingsList
│   └── AIAnalysisCard (run-level summary)
```

- └─ ReportSection
 - └─ ReportSummary
 - └─ DownloadButton

Cross-references: 03-UI-UX-Specification.md (visual treatment of all states/badges used here), 05-Frontend-API-Integration.md (result/AI-analysis/report endpoints), 06-Frontend-State-and-Data-Models.md (TestResult, AIAalysis, Report types).